



UNIVERSITÀ DEGLI STUDI DI PALERMO

DIPARTIMENTO DI INGEGNERIA

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

Object Design Document

SPECIFICHE DEI REQUISITI SOFTWARE

ArtID



Team di Sviluppo

Andrea Sciortino

Davide Alaimo

Gabriele Iovino

Giovanni Luca Cusano

Supervisore

Prof.ssa Valeria Seidita

Anno Accademico

2025 - 2026

Indice Generale

1	Introduzione	2
1.1	Prestazioni e costi	2
1.2	Tempi di risposta	2
1.3	Overview del sistema	2
1.4	Sviluppo Front-End	3
1.4.1	Librerie Front-End	3
1.5	Sviluppo Back-End	3
1.5.1	Librerie Back-End	3
1.6	Linee guida per la documentazione delle interfacce	4
1.7	Linee guida per la documentazione del DBMS	4
1.8	Linee guida per la documentazione dello storage dei file	5
1.9	Convenzioni di Nomenclatura e Mappatura delle Entità	5
2	Packages	7
2.1	artid-client/src	8
2.1.1	routes	8
2.1.2	lib	9
2.2	java/afam/artid-server	10
2.2.1	controller	10
2.2.2	dao	10
2.2.3	model	10
2.2.4	service	10
2.2.5	storage	10
2.2.6	security	10
3	Overview delle classi	12
3.1	Autenticazione	12
3.2	Gestione Profilo	13
3.3	Gestione Materiali	14
3.4	Condivisione	15
3.5	Discovery	17
3.6	DAO	18
3.7	Entity	19

1 Introduzione

1.1 Prestazioni e costi

Per la realizzazione di ArtID sono state impiegate esclusivamente **tecnologie e librerie open source**, minimizzando i tempi e i costi di sviluppo e di esercizio e ottenendo al contempo prestazioni adeguate al dominio applicativo.

A differenza di sistemi che richiedono hardware specializzato, ArtID è pensata come **applicazione web** e non necessita di risorse di calcolo dedicate: il carico è dominato da operazioni di I/O verso il database e verso lo storage. Il livello di Front-End è distribuibile su piattaforma serverless (Vercel, tramite l'adapter dedicato), il livello di Back-End gira su una JVM standard e la persistenza relazionale è affidata a PostgreSQL; i contenuti multimediali sono delegati al servizio di object storage AWS S3, evitando i costi e la complessità di un'infrastruttura di archiviazione proprietaria.

1.2 Tempi di risposta

I tempi di risposta delle operazioni CRUD sono nell'ordine di pochi secondi e tendono a crescere con l'aumentare dei dati nel database; sono mitigati da indicizzazione, conteggi aggregati e pooling delle connessioni. I tempi di caricamento e download dei file dipendono dalla loro dimensione e dal round-trip verso l'object storage; per il download dei file multimediali si impiegano URL firmati (presigned), così che i byte transitino direttamente tra browser e storage senza gravare sul nodo Back-End. ArtID non presenta latenze imprevedibili: tutte le operazioni hanno un costo sostanzialmente deterministico, funzione della dimensione dei contenuti.

1.3 Overview del sistema

Il sistema è strutturato secondo una scomposizione fisica e logica in due macro-package principali distribuiti su un'architettura Client-Server a 5 livelli (*5-Tier Architecture*), che evolve e mappa i concetti del pattern *Entity-Control-Boundary* definito in fase di analisi:

- **Front-End (Nodo Client):** Sviluppato mediante il framework *SvelteKit*, ospita i primi due livelli dell'architettura:
 - *Presentation Layer (Boundary):* Gestisce il rendering delle interfacce grafiche e la cattura degli eventi utente.
 - *Controller Layer - Client:* Implementa una prima logica di controllo decentralizzata, deputata alla gestione dello stato locale dell'applicazione, al routing e alla validazione formale immediata dell'input, al fine di prevenire l'invio di richieste inconsistenti verso il server.
- **Back-End (Nodo Server):** Sviluppato mediante il framework *Java Spring Boot*, centralizza la logica di business e la persistenza dei dati, articolandosi in tre livelli logici:
 - *REST Controller Layer:* Espone le API RESTful del sistema, intercettando i payload HTTP/HTTPS provenienti dal client e validandone la struttura formale.
 - *Service Layer (Control centrale):* Custodisce i componenti di controllo centrali in cui risiede la core business logic del sistema (es. gestione delle transazioni, orchestrazione dei flussi di autenticazione e vincoli di integrità).
 - *DAO Layer (Data Access Object / Repository):* Incapsula i pattern di persistenza delle *Entity* di dominio, interfacciandosi direttamente con il *DBMS*. **In questo livello, l'adozione dell'architettura *Repository* (tramite Spring Data) si giustifica per elevare il livello di astrazione, permettendo al Service Layer di trattare i dati come collezioni di oggetti in memoria ed isolando completamente la logica di business dai dettagli implementativi del database relazionale.**

1.4 Sviluppo Front-End

L'interfaccia di ArtID privilegia semplicità e immediatezza: grazie a un layout lineare, essenziale e responsive, consente la migliore interazione possibile anche a un'utenza priva di competenze tecniche avanzate. Ove possibile, errori e messaggi rivolti all'utente sono resi non bloccanti — tramite notifiche toast — per non interrompere il flusso di lavoro.

L'applicazione è sviluppata con **SvelteKit** (Svelte 5, modalità Runes) e **Bootstrap**, con rendering ibrido lato server (SSR) e lato client (CSR). SvelteKit adotta un modello di routing e rendering basato su file.

1.4.1 Librerie Front-End

Libreria	Versione	Ruolo
svelte	5.x	Libreria UI a componenti reattivi (modalità Runes).
@sveltejs/adaptor-vercel	6.x	Adapter di build per il deployment su Vercel.
vite	8.x	Build tool e dev server con Hot Module Replacement.
bootstrap	5.3	Framework CSS responsive.
@sveltestrap/sveltestrap	7.x	Componenti Bootstrap idiomatici per Svelte.
bootstrap-icons	1.13	Set di icone vettoriali.
sass	1.x	Preprocessore CSS.
zod	4.x	Validazione e derivazione dei tipi a runtime (schemi di dominio e DTO).
openapi-fetch	0.17	Client HTTP tipizzato sullo schema OpenAPI del back-end.
openapi-typescript	7.x	Generazione dei tipi TypeScript dallo schema OpenAPI (dev).
quill	2.x	Editor di testo ricco per le descrizioni dei materiali.
svelte-sonner	1.x	Notifiche toast non bloccanti.

1.5 Sviluppo Back-End

Lo sviluppo lato server adotta **Spring Boot 4** su piattaforma **Java 17**, strutturandosi secondo un'architettura a strati (**Controller** → **Service** → **DAO**) che separa nettamente l'esposizione delle API, la logica di business e l'accesso ai dati.

Le API sono di tipo REST e *stateless*: ogni richiesta viene autenticata in modo indipendente tramite token **JWT** (JSON Web Token), abilitando la scalabilità orizzontale del Back-End del sistema. La persistenza relazionale è gestita in modo snello con **Spring Data JDBC** e, dove si rendono necessarie query o aggregazioni puntuali, tramite l'ausilio di `JdbcTemplate` e `NamedParameterJdbcTemplate`; l'archiviazione dei contenuti binari e dei file multimediali è invece delegata a un **object storage S3-compatibile**.

La specifica **OpenAPI** (generata mediante *springdoc-openapi*) costituisce il contratto formale e condiviso dal quale il client genera programmaticamente i propri tipi e modelli di dato, garantendo una coerenza *end-to-end* tra i due nodi. La sicurezza dell'applicazione è interamente governata da **Spring Security** tramite un filtro di intercettazione per i token JWT e l'algoritmo **BCrypt** per la codifica delle password, mentre l'utilizzo di *Lombok* riduce drasticamente il codice boilerplate all'interno delle classi di dominio.

1.5.1 Librerie Back-End

Libreria	Versione	Ruolo
----------	----------	-------

spring-boot-starter-web	4.0.6	Dispatching delle richieste HTTP sul REST Controller Layer tramite l'infrastruttura di Spring MVC, server embedded Tomcat e serializzazione JSON.
spring-boot-starter-data-jdbc	4.0.6	Repository e mapping relazionale (Spring Data JDBC).
spring-boot-starter-jdbc	4.0.6	JdbcTemplate, datasource e gestione delle transazioni.
spring-boot-starter-security	4.0.6	Autenticazione, autorizzazione e catena di filtri.
spring-boot-starter-mail	4.0.6	Servizio di astrazione per la composizione e l'invio asincrono di e-mail.
postgresql	runtime	Driver JDBC per PostgreSQL.
jjwt (api/impl/jackson)	0.12.6	Generazione e verifica dei token JWT.
springdoc-openapi-starter-webmvc-ui	2.8.8	Esposizione della specifica OpenAPI e Swagger UI.
software.amazon.awssdk:s3	2.31	Client S3 per l'object storage (Supabase Storage).
lombok	—	Riduzione del boilerplate (@Data, @RequiredArgsConstructor).
spring-boot-devtools	runtime	Hot reload in fase di sviluppo.

1.6 Linee guida per la documentazione delle interfacce

Le interfacce sono realizzate con **SvelteKit** e **Bootstrap**, con l'obiettivo di offrire schermate essenziali, coerenti e immediate.

La componentizzazione (organizzata nella cartella `lib/components` e suddivisa in primitive UI, componenti di pagina ed elementi di layout) favorisce il riuso e l'uniformità visiva, mentre l'impiego di stili condivisi assicura un design moderno e responsive su dispositivi desktop.

1.7 Linee guida per la documentazione del DBMS

Il DBMS scelto è **PostgreSQL**, erogato nel deployment corrente come servizio gestito sulla piattaforma **Supabase**. Il database garantisce intrinsecamente il pieno rispetto delle proprietà **ACID** (Atomicity, Consistency, Isolation, Durability) per tutte le transazioni di sistema.

Le caratteristiche principali legate alla gestione e alla sicurezza dei dati includono:

- **Interazione e Sicurezza:** L'interfacciamento con il database avviene mediante il framework *Spring Data JDBC*. L'esecuzione di query unicamente parametrizzate (precompilate tramite `NamedParameterJdbcTemplate`) azzerava completamente il rischio di vulnerabilità da *SQL Injection*.
- **Controllo degli Accessi ed Isolamento:** Il livello di persistenza è rigorosamente segregato. L'accesso al database è mediato in via esclusiva dal nodo *Back-End*, il quale si connette tramite credenziali dedicate e cifrate. Il client non possiede alcun canale di comunicazione diretto verso il DBMS, mitigando i vettori di attacco esterni.
- **Prevenzione degli Accessi Indiretti (IDOR):** Si evidenzia una scelta di progettazione di sicurezza non visibile nei diagrammi di sequenza del RAD (dove le firme dei metodi includono solo gli ID delle risorse, es. `idArtid`). Nel codice effettivo, ogni operazione di lettura o scrittura sul database (es. eliminazione o dettaglio) riceve sempre l'ID del membro autenticato (`@AuthenticationPrincipal`).

iniettato automaticamente tramite JWT. Il *Service Layer* propaga questo ID al database per validare la reale proprietà della risorsa prima di eseguire la query, garantendo a runtime la protezione da vulnerabilità di tipo *Insecure Direct Object Reference*.

I parametri di connessione e autenticazione verso l'istanza remota di Supabase sono centralizzati interamente all'interno del file `application.properties` del backend (`src/main/resources/`). Questa scelta garantisce il totale disaccoppiamento tra la logica dei componenti *DAO* e l'infrastruttura di rete.

Per il dettaglio dello schema entità-relazione (ER) e del relativo modello relazionale si rimanda alle sezioni dedicate all'interno del *System Design Document* (§3.6).

1.8 Linee guida per la documentazione dello storage dei file

Per la persistenza dei contenuti binari (materiali, attestati e immagini di profilo) è impiegato un servizio di **object storage S3-compatibile** — nel deployment corrente **Supabase Storage** — interrogato tramite l'*AWS SDK for Java v2*. La comunicazione avviene su HTTPS verso un servizio specializzato esterno, mantenendo il sistema modulare e scalabile e sgravando il DBMS dalla gestione dei BLOB.

Gestione e Sicurezza dei File

- **Identificazione:** Ogni file è identificato da una *object key* opaca (UUID) scollegata dal nome originale, così da evitare collisioni e *path traversal*.
- **Metadati:** Le informazioni del file (nome, estensione, tipo MIME, dimensione) risiedono nella tabella `file`, mentre i byte sono conservati nel bucket.
- **Accesso:** Il download di oggetti privati avviene tramite URL firmati a scadenza short-term.
- **Disaccoppiamento:** L'intera interazione è incapsulata nella classe `StorageService`, garantendo che un eventuale cambio di provider non impatti la logica di business.
- **Consistenza:** La pulizia degli oggetti è coordinata con le transazioni del database. Un upload viene rimosso in caso di rollback e la cancellazione fisica avviene solo a commit avvenuto, evitando file orfani o riferimenti incoerenti.

1.9 Convenzioni di Nomenclatura e Mappatura delle Entità

Al fine di massimizzare la chiarezza descrittiva e garantire la piena tracciabilità tra la fase di analisi concettuale (RAD) e l'effettiva implementazione software, nel presente documento è stata adottata una convenzione linguistica ibrida:

- **Analisi e Diagrammi:** I componenti di analisi (*Boundary, Control, Entity*), le descrizioni dei casi d'uso e i messaggi scambiati all'interno dei diagrammi di sequenza utilizzano la **lingua italiana** per facilitare la comprensione concettuale dei flussi.
- **Codice Sorgente:** L'implementazione effettiva del software (classi, firme dei metodi, rotte API e tabelle del DBMS) è interamente sviluppata in **lingua inglese**, in conformità con le *best practice* globali dell'ingegneria del software.

Di seguito viene riportata la corrispondenza terminologica tra le entità descritte nel modello di analisi e le rispettive classi Java/tabelle del database relazionale:

Oggetti Entità (RAD)	Classe Java / Tabella DB (Codice)
Membro	User
ArtID	Artid

Materiale	Resource / File <i>(Scissa tra i metadati logici della risorsa e l'asset fisico su Supabase S3)</i>
Certificato	Certification
Token	ExternalShare
Condivisione	InternalShare

2 Packages

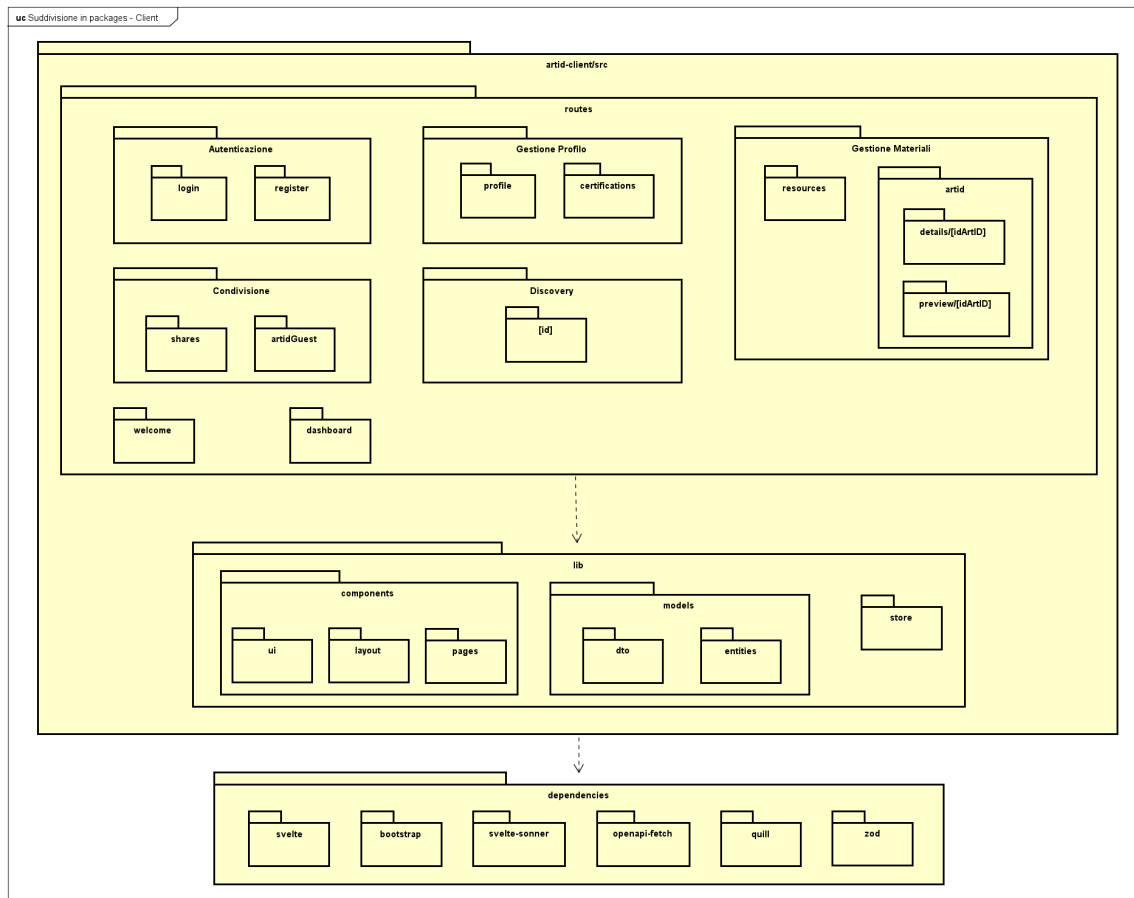


Figure 1: Suddivisione in packages - Client

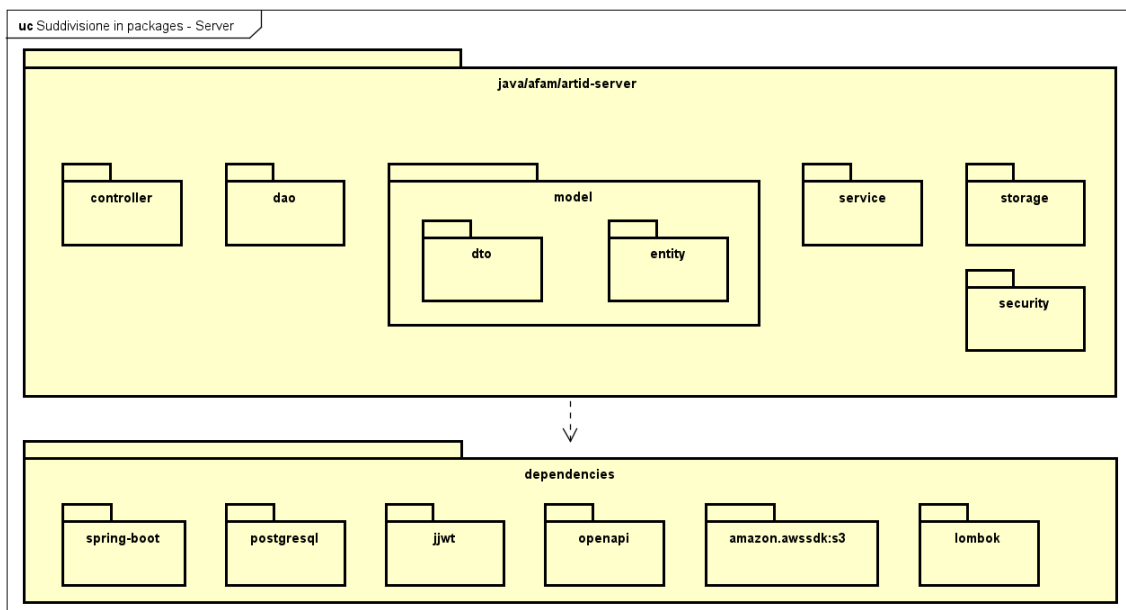


Figure 2: Suddivisione in packages - Server

2.1 artid-client/src

Questa cartella contiene il **Front-end** dell'applicazione.

La struttura sfrutta il routing basato sul file system e la separazione netta tra l'interfaccia grafica e la logica di caricamento dati.

2.1.1 routes

Questa cartella definisce l'albero dei **percorsi navigabili** dell'applicazione; secondo le convenzioni di SvelteKit, ad ogni sottocartella corrisponde un URL accessibile dal browser.

Al fine di mantenere la coerenza con i requisiti del RAD e i diagrammi dell'SDD, è stato introdotto un layer di cartelle fittizie che non alterano il routing, ma servono a raggruppare logicamente i percorsi in base ai relativi macro-casi d'uso.

Il **punto di accesso** principale all'applicazione è definito nella cartella:

- **welcome**: contiene il file `+page.svelte` che implementa la GUI della Landing Page e fornisce i controlli di navigazione iniziali verso le sezioni di autenticazione.

La pagina principale dopo l'autenticazione si trova nella cartella:

- **dashboard**: contiene il file `+page.svelte` che implementa l'interfaccia di controllo centrale, esponendo i collegamenti rapidi a tutte le sezioni funzionali dell'applicazione.

Di seguito vengono dettagliati i percorsi raggruppati per macro-casi d'uso:

2.1.1.1 Autenticazione

Area dedicata alla gestione dell'accesso e dell'identità:

- **login**: composta da `+page.svelte` per la GUI del form e `+page.server.ts` per la gestione sicura delle richieste di accesso lato server e l'integrazione con lo SPID.
- **register**: composta da `+page.svelte` (interfaccia di registrazione) e `+page.server.ts` (validazione e invio dei dati di iscrizione al back-end).

2.1.1.2 Gestione Profilo

Area che permette al Membro di amministrare le proprie informazioni personali:

- **profile**: formata da `+page.svelte` e `+page.server.ts`; consente la modifica dei dati del Membro, il cambio password, il collegamento dello **SPID** e la disattivazione dell'account.
- **certifications**: formata da `+page.svelte` e `+page.server.ts`; fornisce gli strumenti per il caricamento, la visualizzazione e la gestione delle certificazioni professionali del Membro.

2.1.1.3 Gestione Materiali

Area fulcro dell'applicazione per la manipolazione degli asset digitali:

- **resources**: formata da `+page.svelte` e `+page.server.ts`; permette la gestione CRUD (caricamento, rimozione, download e modifica) dei Materiali del Membro.
- **artid**: formata da `+page.svelte` e `+page.server.ts`; dedicata alla creazione guidata e alla consultazione dell'elenco dei propri **ArtID**.
- **artid/details/[idArtID]**: percorso dinamico formato da `+page.svelte` e `+page.server.ts` per l'ispezione approfondita e la consultazione dei metadati di uno specifico **ArtID**.

- `artid/preview/[idArtID]`: percorso dinamico formato da `+page.svelte` e `+page.server.ts` che genera un'anteprima dell'**ArtID**.

2.1.1.4 Condivisione

Sezione predisposta per il tracciamento e il trasferimento sicuro dei dati:

- `shares`: formata da `+page.svelte` e `+page.server.ts`; permette al Membro di monitorare, revocare o generare nuovi link e permessi di condivisione.
- `artIDGuest`: formata da `+page.svelte` e `+page.server.ts`; rappresenta la vista pubblica o ad accesso limitato dedicata ai soggetti esterni che consultano un **ArtID** condiviso.

2.1.1.5 Discovery

Sezione pubblica per la ricerca dei Membri:

- `explore`: formata da `+page.svelte` e `+page.server.ts`; implementa i filtri e il motore di ricerca globale per trovare i profili dei Membri all'interno della piattaforma.
- `explore/[id]`: percorso dinamico formato da `+page.svelte` e `+page.server.ts`; permette la visualizzazione in sola lettura del profilo pubblico di un Membro selezionato.

2.1.2 lib

Questa cartella agisce da contenitore per tutte le risorse condivise, le logiche riutilizzabili e i moduli trasversali dell'applicazione front-end. Al suo interno risiedono anche i modelli TypeScript speculari a quelli definiti nel server Java.

2.1.2.1 components

Raccolta dei componenti dell'interfaccia utente, organizzati secondo principi di riusabilità:

- `layout`: contiene i componenti macroscopici e strutturali dell'applicazione, come la barra di navigazione globale `artid-navbar.svelte`.
- `pages`: ospita componenti complessi dedicati a specifiche schermate, isolati per alleggerire il codice dei file `+page.svelte` principali.
- `ui`: collezione di componenti atomici e privi di stato (stateless), riutilizzabili in qualsiasi sezione del software, come il pulsante stilizzato `artid-button.svelte`.

2.1.2.2 models

Gestisce la controparte client delle strutture dati del database:

- `dto`: definisce le interfacce TypeScript che ricalcano l'esatto formato dei messaggi scambiati tramite chiamate **API REST** con il **back-end**.
- `entities`: definisce i modelli ad oggetti in ambiente TypeScript (`.ts`) che rappresentano la struttura logica delle entità memorizzate nel sistema.

2.1.2.3 stores

Questa cartella si occupa della gestione dello stato globale dell'applicazione. Centralizza le informazioni accessibili da più componenti:

- `auth.ts`: mantiene in memoria l'istanza e i dettagli del Membro correntemente autenticato, esponendo i metodi globali per verificare i permessi di sessione e lo stato dell'autenticazione in tempo

reale.

2.2 java/afam/artid-server

Questo package costituisce il nucleo del **Back-end** dell'applicazione, suddiviso nelle seguenti componenti.

2.2.1 controller

Questa cartella contiene i componenti **REST Controller** responsabili di esporre gli endpoint **API** verso il **front-end**. Il loro compito principale è intercettare le richieste HTTP, validare i dati in ingresso e delegare l'esecuzione della logica di business al rispettivo **service**. Inoltre, i **Controller** si occupano di mappare e convertire i risultati ricevuti dal service nei relativi **dto** (Data Transfer Object) prima di impacchettare e restituire la risposta HTTP al front-end.

2.2.2 dao

Contiene le interfacce **DAO** (Data Access Object) responsabili dell'interazione diretta con il database relazionale. Sfruttando le specifiche di **Spring Data JDBC** tramite l'estensione di **ListCrudRepository**, è sufficiente dichiarare il prototipo del metodo nella firma dell'interfaccia per demandare al framework la generazione automatica della query SQL a runtime. Tuttavia, per operazioni complesse o manipolazioni custom, il sistema consente la scrittura di query personalizzate sfruttando l'annotazione **@Query**.

2.2.3 model

Questo package definisce le strutture dati utilizzate all'interno dell'applicazione ed è suddiviso in due sottopackage principali:

2.2.3.1 dto

Contiene i **Data Transfer Object** (DTO), ovvero classi Java fittizie utilizzate esclusivamente per **modellare** e **standardizzare** il formato dei messaggi e dei payload scambiati nelle richieste e risposte HTTP tra il front-end e il back-end.

2.2.3.2 entity

Contiene le classi Java che rappresentano l'esatta mappatura delle tabelle presenti nel Database. Queste classi sono modellate secondo i requisiti di **Spring Data JDBC** (utilizzando le annotazioni del pacchetto `org.springframework.data.relational.core.mapping`) e sfruttano la libreria Lombok (che tramite l'annotazione **@Data** genera automaticamente a tempo di compilazione tutti i metodi getter, setter, costruttori e codice boilerplate equivalente) per massimizzare la leggibilità del codice.

2.2.4 service

Questa cartella contiene le classi in cui è implementata la **logica di business** dell'applicazione. I componenti di questa cartella orchestrano il flusso dei dati interni: ricevono gli input dai **controller**, interrogano o modificano lo stato del database invocando i relativi **dao**, applicano le regole applicative e i controlli di consistenza, e infine restituiscono gli oggetti di business o le entità elaborate direttamente ai **controller**.

2.2.5 storage

Questo package gestisce la persistenza degli asset digitali (Materiali, allegati e foto profilo) sfruttando un'infrastruttura di Object Storage esterna. Contiene le classi di configurazione e i servizi per l'interazione con endpoint S3-compatibili.

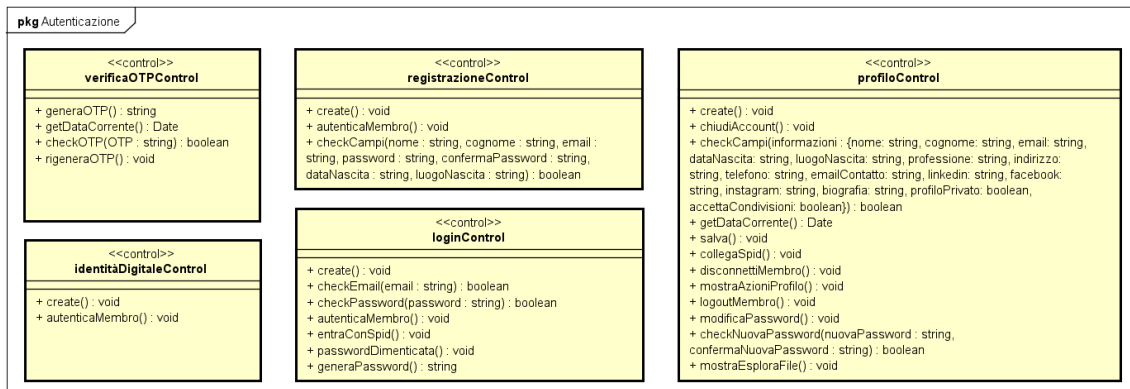
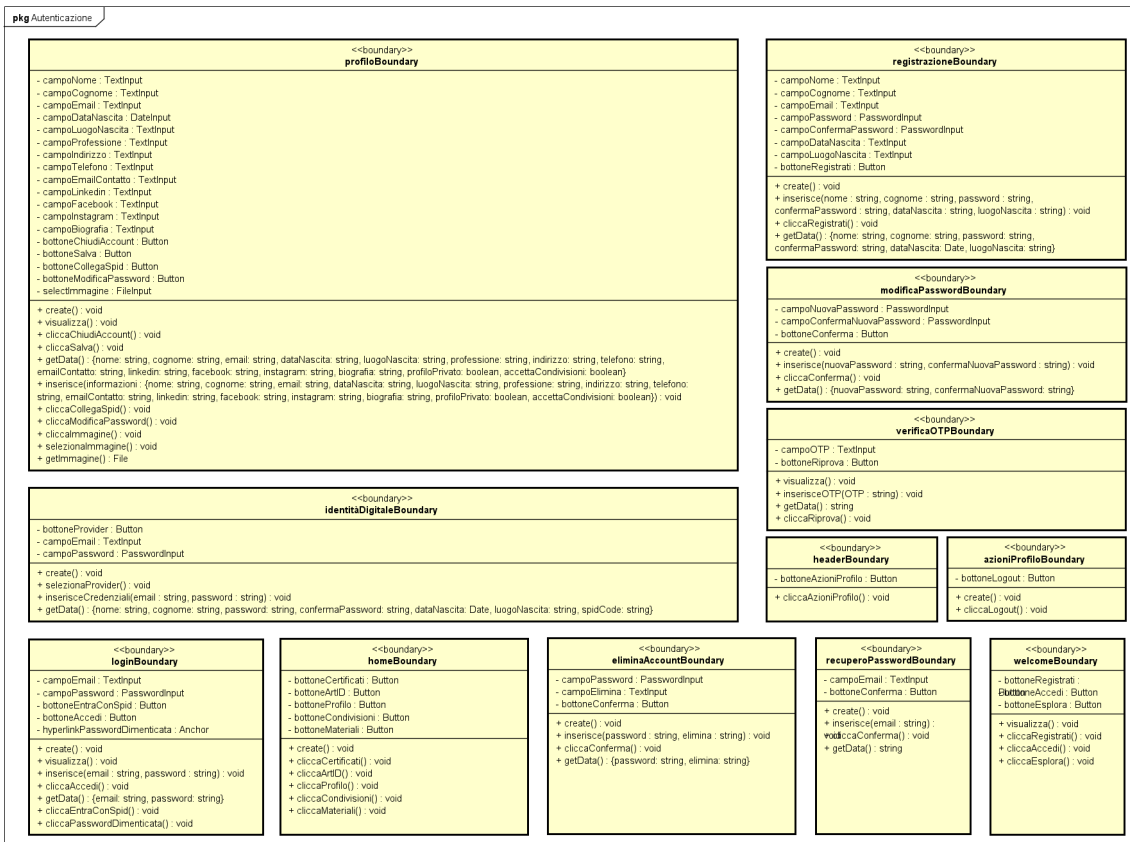
2.2.6 security

Questo package centralizza l'intera infrastruttura di autenticazione e controllo degli accessi dell'applicativo basata su **Spring Security**. Configurato in modalità rigorosamente **stateless** tramite l'utilizzo di token **JWT**, gestisce il ciclo di vita delle sessioni utente senza sovraccaricare il server.

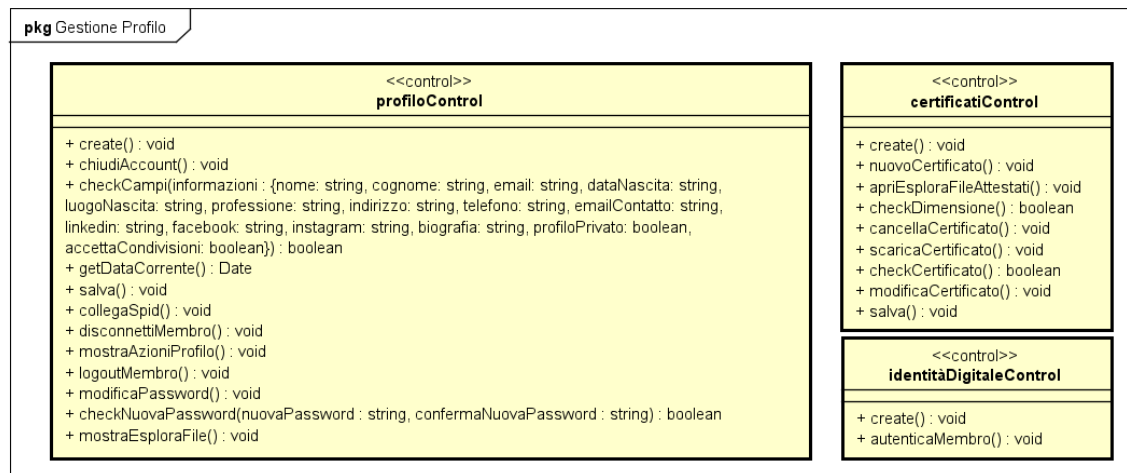
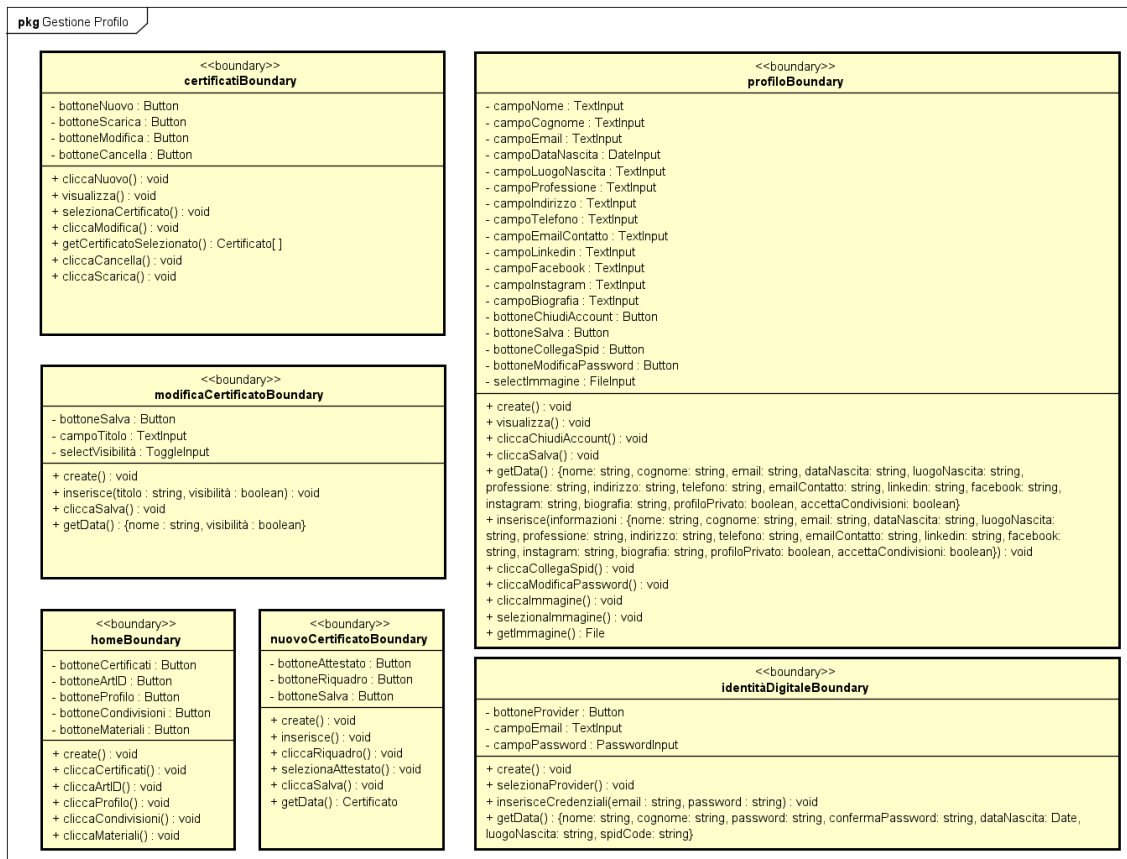
Al suo interno risiedono le componenti per la generazione e validazione crittografica dei token JWT, il filtro per l'intercettazione e il controllo delle richieste HTTP, la configurazione della catena di sicurezza con la definizione dei permessi delle rotte.

3 Overview delle classi

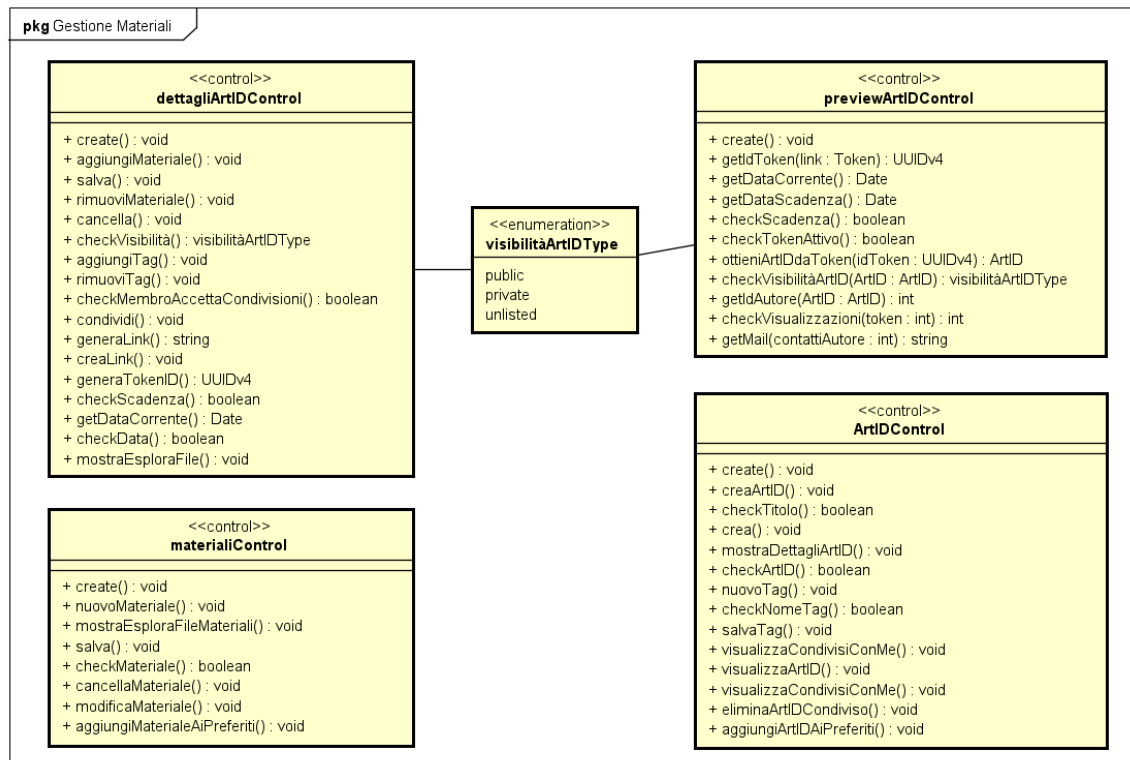
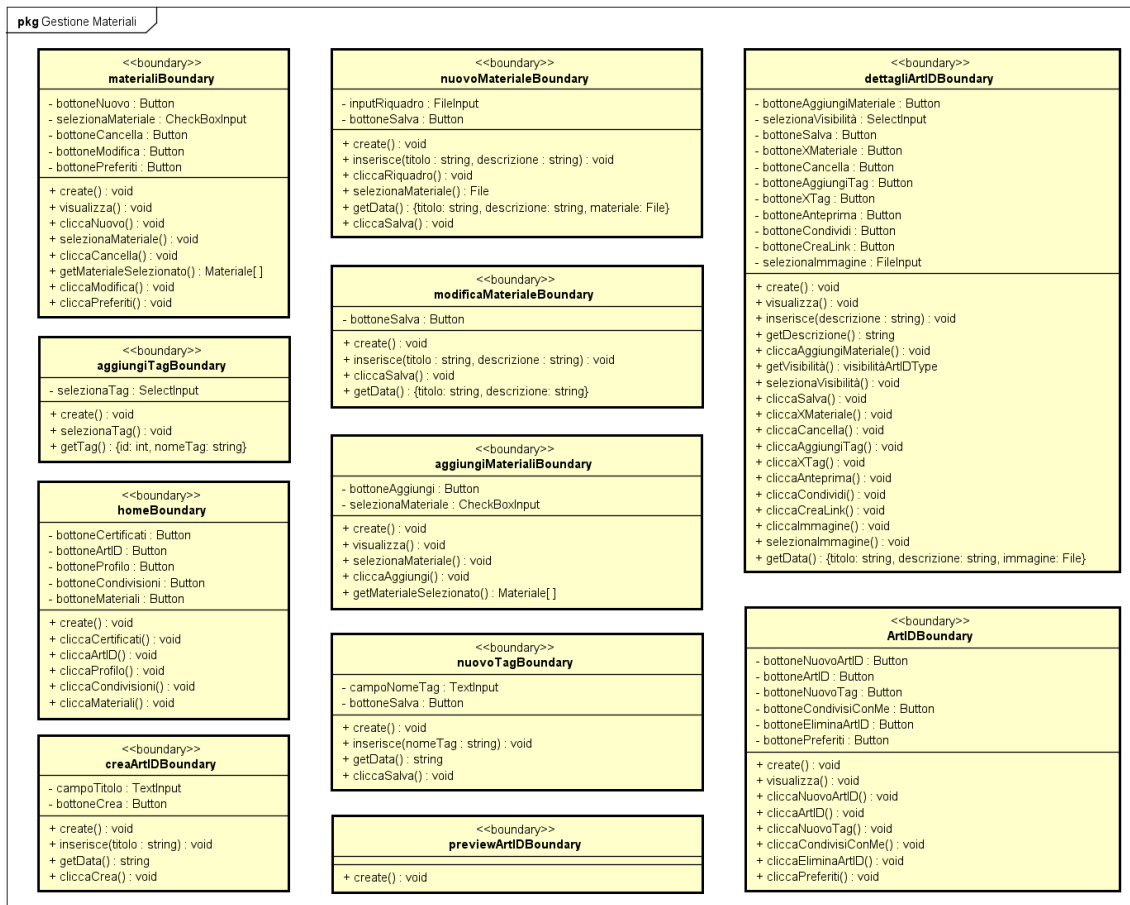
3.1 Autenticazione



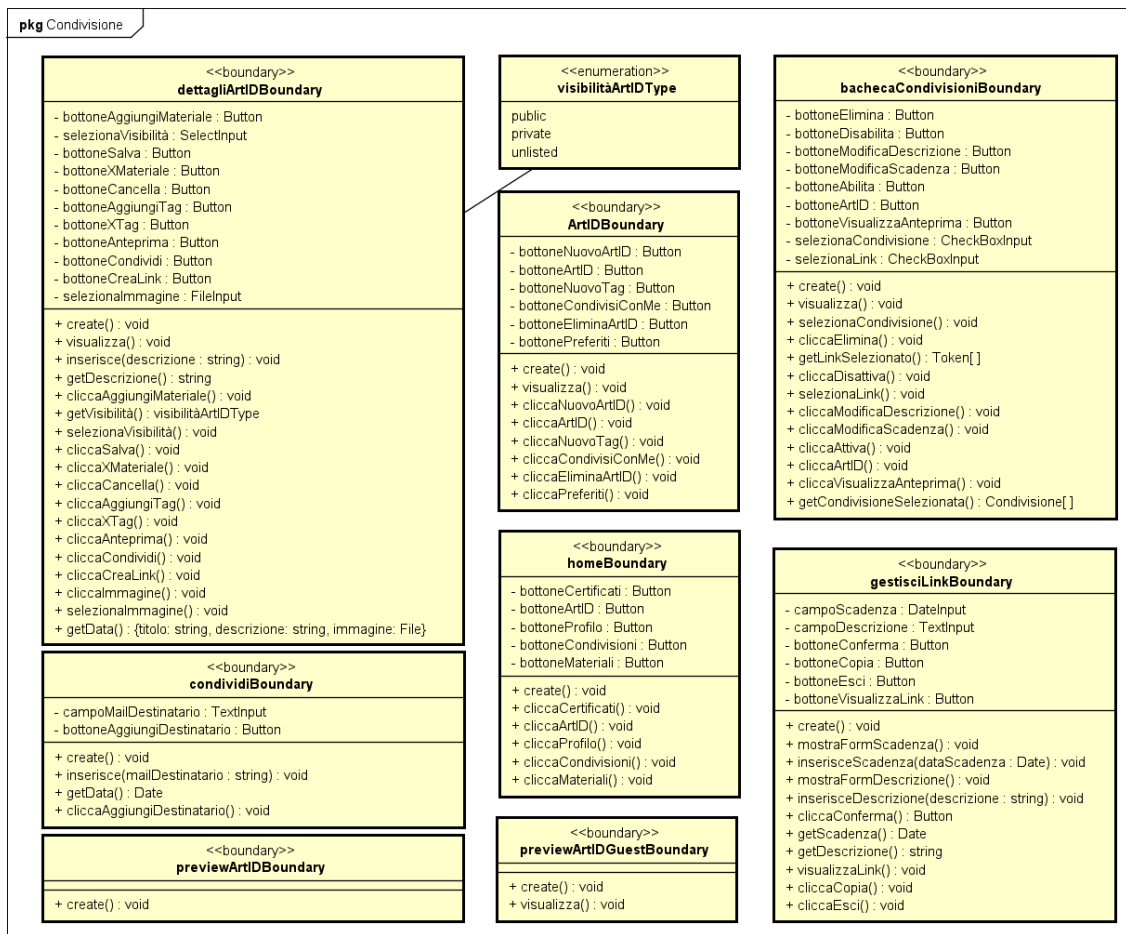
3.2 Gestione Profilo

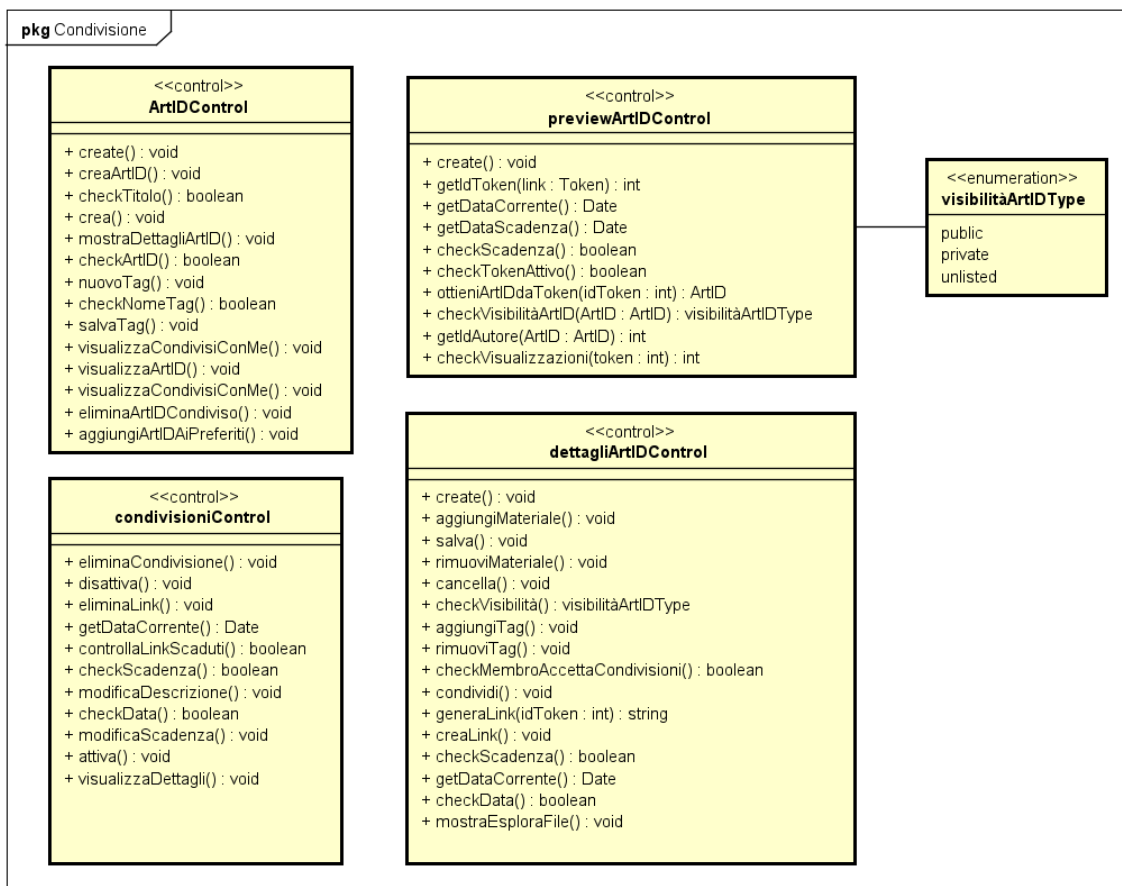


3.3 Gestione Materiali

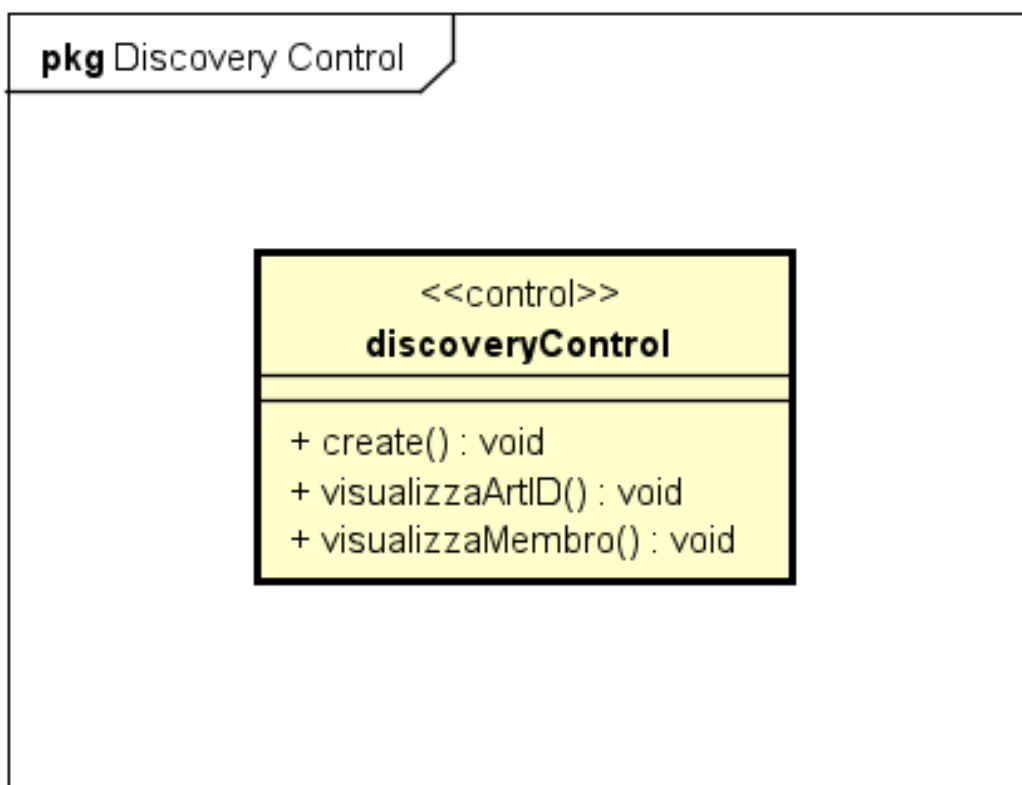
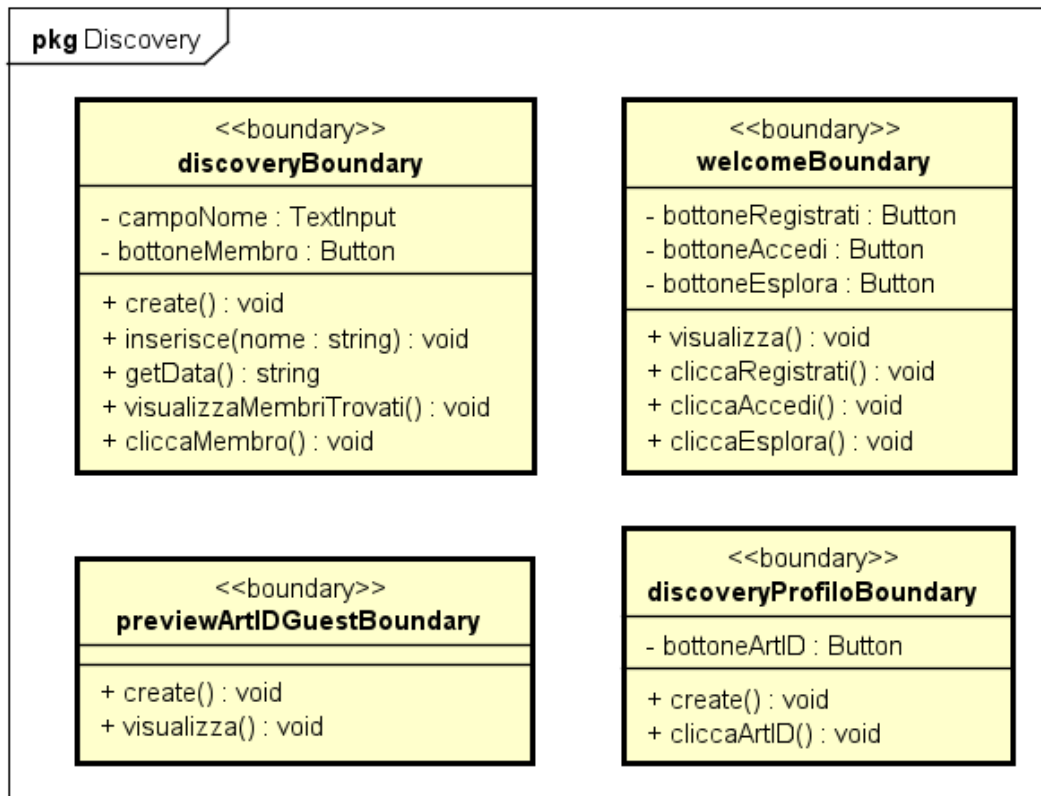


3.4 Condivisione





3.5 Discovery



3.6 DAO

pkg DAO

<<boundary>>
DBMSBoundary

- checkEsistenzaMail(email: string): boolean
- salvaMembro(nome: string, cognome: string, email: string, password: string, dataNascita: string, luogoNascita: string): void
- ottieniDataMembro(email: string, password: string): (nome: string, cognome: string, email: string, dataNascita: string, luogoNascita: string, professione: string, indirizzo: string, telefono: string, emailContatto: string, linkedin: string, facebook: string, instagram: string, biografia: string, profiloPrivate: boolean, accettaCondivisioni: boolean)
- checkEsistenzaSpiCodeSpiCode(int): boolean
- salvaMembroConSpiCodeSpiCode(Membro: (nome: string, cognome: string, email: string, dataNascita: Date, luogoNascita: string, spiCode: int): void
- associaSpiCodeMembro: (nome: string, cognome: string, email: string, dataNascita: Date, luogoNascita: string, spiCode: int): void
- salvaOTPI(email: string, OTP: int): void
- aggiornaPassword(Membro: int, nuovaPassword: string): void
- checkPassword(Membro: int, password: string): boolean
- eliminaContatto(Membro: int): void
- ottieniMateriale(Membro: int): Materiale[]
- cancellaMateriale(Membro: int, titolo: string, descrizione: string, materiale: File): void
- cancellaMateriale(Materiale: int): void
- aggiornaMateriale(Materiale: int, titolo: string, descrizione: string): void
- aggiornaMaterialePreferito(Materiale: int, preferito: boolean): void
- ottieniArti(Membro: int): Arti[]
- creaArti(Membro: int, titolo: string): void
- aggiornaArti(Membro: int, idMateriale: int): void
- aggiornaArtiPreferito(Membro: int, preferito: boolean): void
- aggiornaVisibilitaArti(Membro: int, visibilita: visibilitaArtiType): void
- aggiornaDescrizioneArti(Membro: int, descrizione: string): void
- ottieniDettaglioArti(Membro: int, titolo: string, descrizione: string, visibilita: visibilitaArtiType, materiale: Materiale[]): void
- rimuoviMateriale(Membro: int, idMateriale: int): void
- aggiornaInformazioniArti(Membro: int, titolo: string, descrizione: string, immagine: File): void
- cancellaArti(Membro: int): void
- checkArtiCondivisioni(Membro: int): boolean
- ottieniTag(Membro: int, idTag: int, nomeTag: string): void
- creaTag(Membro: int, nomeTag: string): void
- aggiornaTag(Membro: int, idTag: int, nomeTag: string): void
- rimuoviTag(Membro: int, idTag: int): void
- ottieniContatti(Membro: int): (nome: string, cognome: string, emailContatto: string, linkedin: string, instagram: string, facebook: string)
- aggiorniInformazioniMembro(Membro: int): (nome: string, cognome: string, email: string, dataNascita: string, luogoNascita: string, professione: string, indirizzo: string, telefono: string, emailContatto: string, linkedin: string, facebook: string, instagram: string, biografia: string, profiloPrivate: boolean, accettaCondivisioni: boolean)
- aggiorniImmagineProfilo(Membro: int, immagine: File): void
- ottieniCertificati(Membro: int): Certificati[]
- salvaCertificato(Membro: int, titolo: string, visibilita: boolean, attestato: File): void
- cancellaCertificato(Certificato: int): void
- scaricaCertificato(Certificato: int): void
- aggiornaCertificato(Certificato: int, titolo: string, visibilita: boolean): void
- ottieniCondivisioneInterna(Membro: int, idDestinatario: string): void
- creaCondivisioneInterna(Membro: int, idDestinatario: string, idDestinatario: int, accettaCondivisioni: boolean): void
- ottieniMembroDaMail(MailDestinatario: string): (nome: string, cognome: string, email: string, dataNascita: string, luogoNascita: string, professione: string, indirizzo: string, telefono: string, emailContatto: string, linkedin: string, facebook: string, instagram: string, biografia: string, profiloPrivate: boolean, accettaCondivisioni: boolean)
- ottieniMembro(Membro: int): string
- disattivaToken(Token: int): void
- ottieniCondivisioneInterna(Membro: int): (condivisioni: Condivisione[], dataArti: Arti[]): void
- rifiutaCondivisioneInterna(Membro: int, idDestinatario: int): void
- cancellaToken(Token: int): void
- salvaToken(dataScadenza: Date, descrizione: string, idArti: int): void
- aggiornaScadenza(Token: int, dataScadenza: Date): void
- aggiornaDescrizioneCondivisione(Token: int, descrizione: string): void
- riattivaToken(Token: int): void
- ottieniContattiMembro(Membro: int): (nome: string, cognome: string, emailContatto: string, linkedin: string, instagram: string, facebook: string)
- aggiornaPrimariaCondivisione(Token: int): void
- ottieniDataToken(Token: int): Token
- aggiornaVisualizzazione(Token: int): void
- ottieniCondivisioneEsterna(Membro: int): (token: Token[], dataArti: Arti[]): void
- ottieniCondivisioneInterna(Membro: int): (condivisioni: Condivisione[], dataArti: Arti[]): void
- ottieniCertificati(Membro: int): Certificati[]
- ottieniDettaglioArti(Membro: int): (nome: string, cognome: string, emailContatto: string, linkedin: string, instagram: string, facebook: string): void
- cancellaMembro(Membro: int): void
- richiediRiconfermaDBMS(): void

<<boundary>>
messageBoundary

- bottoneOk: Button
- bottoneConferma: Button
- visualizzaMessage: string): void
- cliccaOk(): Button
- cliccaConferma(): Button

<<boundary>>
errorBoundary

- visualizzaError: string): void

3.7 Entity

